

Syntax Analyzer for IF-ELSE Statements using Flex and Bison

Aim

- To design a syntax analyzer that accepts valid `if`, `if-else`, and nested `if-else` statements.
- To detect syntax errors in the provided statements.
- To resolve the dangling-else problem using grammar precedence and associativity rules.
- To parse statements interactively via the command line interface.

Software Requirements

Tool	Purpose
Flex	Lexical Analyzer Generator (Tokenizes the input stream)
Bison	Parser Generator (Validates syntax using Context-Free Grammar)
GCC	C Compiler to compile the generated source files
OS	Windows (MinGW environment) / Linux

Theory

- **Flex Component:** Scans the input character stream and converts it into a sequence of tokens such as `IF`, `ELSE`, `ID`, `RELOP`, and `NUM`.
- **Bison Component:** Utilizes a Context-Free Grammar (CFG) to parse the semantic tokens produced by Flex and structurally validate the execution block.
- **Dangling Else Resolution:** The inherent ambiguity of the dangling else problem is gracefully resolved within Bison by declaring token precedence rules (`%nonassoc`) ensuring that an `else` aligns with the nearest open `if`.

Program Structure

1. Lexical Analyzer File: ifelse.l

```
%{
#include "ifelse.tab.h"
%}

%%

"if"          { return IF; }
"else"        { return ELSE; }
"="           { return ASSIGN; }
";"           { return SEMICOLON; }
"("           { return '('; }
")"           { return ')'; }
[<>!=]=?      { return RELOP; }
[0-9]+        { yylval = atoi(yytext); return NUM; }
[a-zA-Z_][a-zA-Z0-9_]* { return ID; }
[
]             { /* skip whitespace */ }
[
]             { return '
'; }
.             { printf("Unknown symbol: %s
", yytext); }

%%

int yywrap() {
    return 1;
}
```

2. Parser File: ifelse.y

```
%{
#include <stdio.h>
#include <stdlib.h>

int yylex(void);
void yyerror(const char *s);
int isError = 0;
%}

%token IF ELSE ID RELOP ASSIGN NUM SEMICOLON
```

```

%nonassoc LOWER_THAN_ELSE
%nonassoc ELSE
%start stmt

%%

stmt : if_stmt '
' {
    if (!isError)
        printf("valid
");
    }
    ;

if_stmt : IF '(' cond ')' stmt_block %prec LOWER_THAN_ELSE
        | IF '(' cond ')' stmt_block ELSE stmt_block
        ;

stmt_block : ID ASSIGN expr SEMICOLON
           | if_stmt
           ;

cond : ID RELOP ID
     | ID RELOP NUM
     ;

expr : ID
     | NUM
     ;

%%

void yyerror(const char *s) {
    isError = 1;
    printf("Syntax Error: Invalid statement
");

    int c;
    while ((c = getchar()) != '
' && c != EOF);
}

int main() {
    printf("=== IF-ELSE Statement Parser ===
");
    printf("Enter one statement per line, eg:
");

```

```
    printf("if (a>b) x=y;\n");\n    printf("if (a>b) x=y; else x=z;\n");\n    printf("Press Ctrl+C or Ctrl+Z (Windows) to exit.\n");\n\n    while (1) {\n        isError = 0;\n        printf("> ");\n        yyparse();\n    }\n    return 0;\n}
```

Compilation and Execution Steps

Execute the following commands sequentially in your terminal/command prompt to build and run the parser:

```
$ bison -d ifelse.y\n$ flex ifelse.l\n$ gcc ifelse.tab.c lex.yy.c -o ifelsechecker\n$ ./ifelsechecker
```